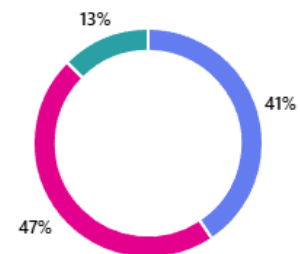


# Föreläsning 260513 – Interface

21. Vad är ett interface?

[Mer](#)

|                                            |    |
|--------------------------------------------|----|
| ● Jag förstår detta bra                    | 13 |
| ● Jag har delvis koll, men behöver öva mer | 15 |
| ● Jag behöver läsa på/förstå bättre        | 4  |



## Interface

```
interface ISumma
{
    double Addera(); // Som en klass, men utan variabler, konstruktör och
                    // metoder
}
```

Ett interface är ett kodblock som bara kan innehålla metod-definitioner och egenskaper utan kod-kroppar. På det sättet påminner det om en abstract klass, men en abstract klass får innehålla medlems-variabler, fullständiga metoder. Interface namn börjar alltid med stora bokstaven **I**.

Ett interface är en typ, och klasser kan ärva från den. Men de ärver inget fysiskt, utan klasserna **måste implementera interface-metoderna**.

- ▶ Liknar klass
- ▶ Ett kontrakt
- ▶ Är alltid public.
  - ▶ Detta ska vi **inte** ange.
- ▶ Implementerar **inte** metoderna.
- ▶ Klasser kan inte ärva från flera klasser (s.k. multipelt arv) men kan ärva från flera interface.

- ▶ Använder vi ett interface ska vi kunna räkna med att de metoder som interfacet har också finns i den klass som ärver från interfacet.
- ▶ Interface betyder gränssnitt.
  - ▶ Ett gränssnitt är ett slags gemensamt beteende.
  - ▶ Bara vi vet hur vi använder gränssnittet kan vi använda något.
  - ▶ Detta något kan vara en bil eller en klass.
- ▶ Man kan säga att ratt och pedaler m.m. är gränssnittet för bilen.
- ▶ Vi lär oss köra bil genom att lära oss växla m.m.
- ▶ Motorn fungerar olika i olika bilar men vi behöver bara veta hur vi kör bilen.
- ▶ På samma sätt kan klasser fungera på olika vis men ha samma gränssnitt.
- ▶ Säg att vi har ett gränssnitt (interface) för att lägga in data
  - ▶ Den har en metod **AddInfo**.
  - ▶ Om en klass använder gränssnittet vet vi att den klassen lägger till data i en metod som heter **AddInfo**.
  - ▶ Exakt hur metoden gör detta spelar ingen roll.
  - ▶ Bara vi vet att vi kan använda den metoden och hur.
- ▶ **När en klass implementerar ett interface så:**
  - ▶ Går man med på villkoren i kontraktet.
  - ▶ Det betyder att man måste implementera alla medlemmar i interfacet.

### Explicit Interface

- ▶ Finns risk att två interface har samma metodsSignatur.
- ▶ Låt säga att vi har ett interface som har en metod som heter run();
- ▶ Ett annat interface har exakt samma metod.

- Om vi då använder båda dessa interface på en klass, hur ska kompilatorn veta vilken vi vill köra?

När en klass implementerar två interface med samma metods signatur (run()), behöver klassen bara **en enda implementation**.

```
interface IRobot { void Run(); }
interface IMotor { void Run(); }

class MinKlass : IRobot, IMotor
{
    public void Run() // EN implementation räcker!
    {
        Console.WriteLine("Kör!");
    }
}
```

- Men om vi vill ha olika beteende, får vi använda Explicit interface-implementering.

```
class MinKlass : IRobot, IMotor
{
    void IRobot.Run() // Specifik för IRobot
    {
        Console.WriteLine("Robot kör!");
    }

    void IMotor.Run() // Specifik för IMotor
    {
        Console.WriteLine("Motor kör!");
    }
}
```

I main:

```
IRobot robot = new MinKlass();

IMotor motor = new MinKlass();
robot.Run(); // "Robot kör!"
motor.Run(); // "Motor kör!"
```